hashlock.

Security Audit

Tons Money (DeFi)

November 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	6
Security Rating	7
Intended Smart Contract Functions	8
Code Quality	9
Audit Resources	9
Dependencies	9
Severity Definitions	10
Status Definitions	11
Audit Findings	12
Centralisation	24
Conclusion	25
Our Methodology	26
Disclaimers	28
About Hashlock	29

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

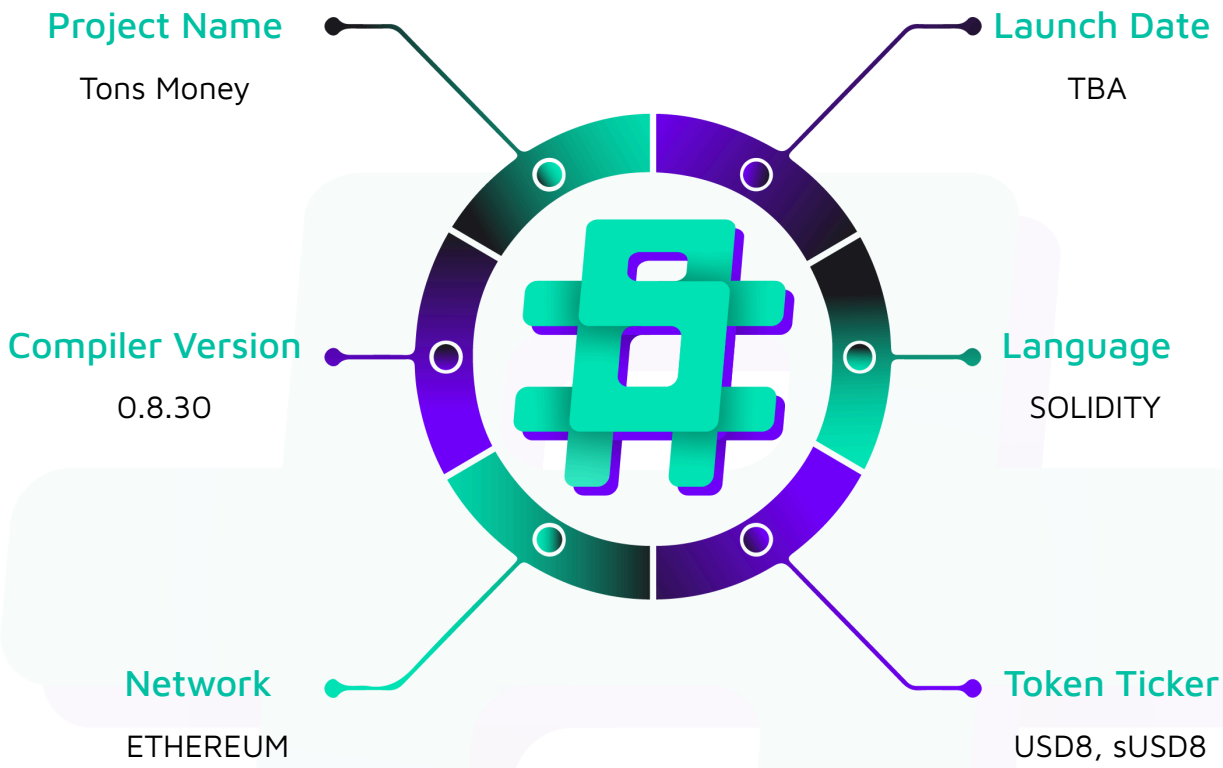
The Tons Money team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Tons Money is a DeFi platform offering a stablecoin called USD8 ("Infinity Dollar"), pegged 1:1 to the US Dollar and fully backed by major reserves (USDC/USDT). The platform enables users to mint USD8, stake it for auto-compounding yield via a derivative token (sUSD8), and benefit from yield-generating strategies powered by blue-chip DeFi protocols—all while emphasizing stability, transparency, and gas-efficiency.

Project Name: Tons Money
Project Type: DeFi
Compiler Version: 0.8.30
Logo:

Visualised Context:



Audit Scope

We at Hashlock audited the solidity code within the Tons Money project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Tons Money Smart Contracts**
Platform	**Ethereum / Solidity**
Audit Date	**November, 2025**
Contract 1	IUSD8 (interface)
Contract 2	Staking.sol
Contract 3	USD8.sol
Contract 4	Vault.sol
Fix Review GitLab Commit Hash	5993d47cff2ab2e32c28eba0532152

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High severity vulnerabilities

1 Medium severity vulnerabilities

4 Low severity vulnerabilities

1 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
IUSD8.sol: - Simple interface: defines the function used in the contract.	**Contract achieves this functionality**
USDC8.sol: - Simple token contract that allows the admin to mint and burn tokens.	**Contract achieves this functionality**
Staking.sol - A staking contract that allows users to stake, unstake, claim, and redeem tokens.	**Contract achieves this functionality**
Vault.sol - Manages deposits and withdrawals of supported stablecoins, converting them to and from the platform's USD8 token using Chainlink price feeds. - It uses role-based access control for admins and operators to manage tokens, treasury, and cooldown settings.	**Contract achieves this functionality**

Code Quality

This audit scope involves the smart contracts of the Tons Money project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Tons Money project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] Vault.sol#_getPrice - Missing staleness and heartbeat validation allows use of outdated or frozen oracle prices

Description

The contract's price retrieval logic does not verify whether the oracle data is fresh or valid before using it. Specifically, the `_getPrice` function reads values from a Chainlink-compatible `AggregatorV3Interface`. Still, it fails to perform safety checks to ensure that the returned price data is recent and that the oracle feed is actively updating.

This omission exposes the protocol to severe risks if the oracle feed becomes stale, stops updating, or is manipulated to freeze a favorable price.

Vulnerability Details

The affected function:

```
function _getPrice(address priceFeed) internal view returns (uint256, uint256) {

    (, int256 price,,,) = AggregatorV3Interface(priceFeed).latestRoundData();

    uint8 decimals = AggregatorV3Interface(priceFeed).decimals();




    return (uint256(price), uint256(decimals));

}
```

The function directly reads the latest round data without validating the following critical conditions:

- `answeredInRound >= roundId` – ensures the round is fully answered.
- `updatedAt != 0` – ensures the round has valid data.
- `block.timestamp - updatedAt <= heartbeat` – ensures the data is recent and not stale.

Without these checks, the function may return old or frozen prices.

If the Chainlink oracle stops updating (e.g., due to network downtime, misconfiguration, or malicious manipulation), the function will continue to return the last known price indefinitely.

An attacker could exploit this by performing time-sensitive actions (such as staking, minting, borrowing, or liquidation) using an outdated but favorable price.

Example Attack Scenario

1. The price feed for an asset stops updating, leaving the last price as $2,000.
2. The real market price drops to $1,200, but the contract still reads $2,000.
3. An attacker calls a function that uses `_getPrice()` to mint or borrow against inflated collateral value, receiving far more tokens or liquidity than should be allowed.
4. Because the contract does not validate data freshness, it accepts the old price as valid, causing permanent financial loss to the protocol.

If the oracle address is immutable (i.e., it cannot be replaced), the system remains stuck with frozen data indefinitely.

Impact

High severity — protocol can be `manipulated` using stale or outdated oracle data.

Users can mint, borrow, or stake using incorrect prices.

Entire pricing-dependent functionality (such as rewards, collateral, or redemptions) becomes unreliable.

No on-chain mechanism exists to detect or recover from a frozen oracle.

Recommendation

Implement full staleness and heartbeat validation before using the returned price.
A secure implementation should check that:

- The price is positive.
- The round is complete (`answeredInRound >= roundId`).
- The price was updated recently (within a defined heartbeat window).

```
function _getPrice(address priceFeed) internal view returns (uint256 price, uint8 decimals) {
    AggregatorV3Interface feed = AggregatorV3Interface(priceFeed);
      (uint80 roundId, int256 answer, , uint256 updatedAt, uint80 answeredInRound) = feed.latestRoundData();

    require(answer > 0, "Invalid price");
    require(answeredInRound >= roundId, "Stale round");
    require(updatedAt > 0, "Incomplete round");

    // Check for heartbeat freshness (e.g., 1 hour)
    uint256 heartbeat = 1 hours;
    require(block.timestamp - updatedAt <= heartbeat, "Stale price data");

    price = uint256(answer);
    decimals = feed.decimals();
}
```

Additionally, store a configurable `heartbeat` value for each feed to adjust tolerance per asset volatility and expected oracle frequency.

Comment

Client is aware that the fix introduced a potential DoS risk if the stablecoin goes outside the expected range, and that they consider the current configuration acceptable for now given the use of top stablecoins and the upgradeable contract.

Status

Resolved

Medium

[M-01] Vault.sol#_getPrice - Unsafe casting of signed price value allows incorrect pricing when a malicious or invalid price feed is used

Description

The contract retrieves price data from a Chainlink-style `AggregatorV3Interface` and casts the signed `int256` price value to an unsigned `uint256` without validation. This is unsafe because a price feed can technically return a negative value — either due to misconfiguration, an oracle malfunction, or if a malicious feed address is mistakenly set. The function `_getPrice` performs a direct cast from `int256` to `uint256` without checking the sign of the returned value.

Vulnerability Details

The `_getPrice` function reads the price and decimals as follows:

```
function _getPrice(address priceFeed) internal view returns (uint256, uint256) {

    (, int256 price,,,) = AggregatorV3Interface(priceFeed).latestRoundData();

    uint8 decimals = AggregatorV3Interface(priceFeed).decimals();



    return (uint256(price), uint256(decimals));

}
```

If `price` is negative, casting to `uint256` will wrap the value around into a very large number, leading to incorrect pricing logic across the protocol.

For example, suppose an incorrect or malicious feed is set that returns `price = -2000`. Casting `int256(-2000)` to `uint256` results in a huge number near `2^256 - 2000`, which could drastically inflate the token value, reward amounts, or collateral ratio computations that depend on `_getPrice()`.

This issue is exacerbated if:

- The `priceFeed` address cannot be updated (as mentioned in another issue).
- The protocol relies on `_getPrice()` for critical calculations such as minting, staking rewards, or collateral valuation.

In such a scenario, an attacker could deliberately set or exploit a misconfigured feed to manipulate protocol behavior in their favor.

Impact

The contract may accept invalid or manipulated prices, leading to incorrect reward, minting, or redemption logic.
An attacker could gain a financial advantage by feeding artificially inflated prices.

Since the oracle address cannot be replaced, the contract becomes permanently compromised once an invalid feed is introduced.

Recommendation

Add a validation check to ensure that the price returned from the feed is positive before casting and using it.

Comment

Client is aware that the fix introduced a potential DoS risk if the stablecoin goes outside the expected range, and that they consider the current configuration acceptable for now given the use of top stablecoins and the upgradeable contract.

Status

Resolved

Low

[L-01] Staking.sol - totalStakedAssets and totalUnstakedAssets do not accurately represent the current total staked amount

Description

The contract maintains two separate tracking variables, `totalStakedAssets` and `totalUnstakedAssets`, intended to represent the total amount of assets currently staked and unstaked in the system. However, the logic used to update these variables does not correctly reflect the net staked balance.

`totalStakedAssets` is only incremented when users stake, but never decremented when they unstake. Conversely, `totalUnstakedAssets` is incremented during unstake operations but never decremented or reconciled with `totalStakedAssets`. As a result, both variables continually increase over time, and neither reflects the current live amount of assets in the staking contract.

Vulnerability Details

On-chain or off-chain integrations relying on these getters may display misleading staking statistics.

Protocol metrics such as TVL, APY, or reward calculations could be based on incorrect data.

The system may appear to have more assets staked than it actually does.

Recommendation

Use a single variable (e.g., `currentTotalStaked`) that increases when users stake and decreases when they unstake, accurately reflecting the current total staked assets.

Status

Acknowledged

[L-02] Staking.sol - Struct Instantiation Inside abi.encode in EIP-712 Typehash

Description

In the current implementation, the EIP-712 struct hash is computed using the following pattern:

Vulnerability Details

In the current implementation, the EIP-712 struct hash is computed using the following pattern:

```
keccak256(
    abi.encode(
        MESSAGE_TYPEHASH,
         SignedAction({account: account, isStake: isStake, amount: amount, price: price, nonce: nonce})
    )
)
```

It is recommended to write the parameters separately instead of passing the struct literal inside `abi.encode`.

Impact

Potential confusion about how EIP-712 struct hashing actually works.

Recommendation

Use explicit field encoding that matches the EIP-712 type definition:

```
bytes32 structHash = keccak256(
    abi.encode(
        MESSAGE_TYPEHASH,
        account,
        isStake,
```

```
        amount,
        price,
        nonce
    )
);
```

Status

Acknowledged

[L-03] Vault.sol - token address is hardcoded

Description

The Vault contract defines a `USDC_ADDRESS` variable with a hardcoded value.

Vulnerability Details

It is not recommended to hardcode token values, especially when tokens are upgradeables.

Recommendation

We recommend setting the `USDC_ADDRESS` value via the constructor. Consider also adding a setter function.

Status

Acknowledged

[L-4] Staking.sol - shares transfer functionality is not disabled.

Description

The contract claims that ERC4626 shares are non-transferable, but the code does not prevent transfers. Shares are minted as ERC20 tokens and inherit standard ERC20 transfer functionality.

Vulnerability Details

While the contract intends to create non-transferable shares, it does not override the `transfer` or `transferFrom` functions of ERC20. As a result, minted shares can be freely transferred between addresses, contradicting the intended non-transferable behavior.

```
function _deposit( address caller, address receiver, uint256 assets, uint256 shares )
internal override { _mint(receiver, shares); // shares are minted as ERC20 }
```

The `_deposit` function mints shares without restricting ERC20 transfers, which allows recipients to transfer their shares freely.

Impact

Users can transfer shares to other addresses, potentially bypassing access restrictions or governance limitations that assume shares are non-transferable.

Recommendation

We recommend overriding the `transfer` and `transferFrom` functions to revert any attempts to transfer shares, ensuring shares remain non-transferable as intended.

Status

Resolved

QA

[Q-01] Vault.sol - Withdrawals will fail if the contract lacks sufficient USDC balance

Description

The `batchClaimWithdraw` function attempts to transfer the total withdrawal amount in USDC to the user after processing their withdrawal requests. However, the contract does not guarantee that enough USDC liquidity is available to fulfill these withdrawals.

If the contract's USDC balance is insufficient, the `safeTransfer` call will revert, preventing users from completing their withdrawal process.

Vulnerability Details

If there is not enough USDC in the contract balance, the `safeTransfer` call will revert, effectively blocking all withdrawals in that transaction.

Impact

Users will be unable to withdraw their funds if the contract does not maintain sufficient USDC liquidity. This could occur even if users have legitimate withdrawal requests, leading to blocked withdrawals and a poor user experience.

Recommendation

This might be the expected behavior. This issue is being reported to highlight that the described scenario is possible to take place

Status

Acknowledged

Centralisation

The Tons Money project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Tons Money project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au